Advanced Programming Exam – January 2026

Questions. Download a zip file with the code setup from LearnIT. Unzip it. Solve the tasks in the file `Exam.scala`. The exam consists of 9 questions to be solved within 4 hours. The questions are independent, in the sense that each can be answered even if the preceding questions had not been answered. However, answering earlier questions may make answering later questions easier.

What can I use? During the exam, you can access any static written materials, printed and online, but you are not allowed to communicate with anybody or with anything (AI bots). Using Copilot, ChatGPT, agentic coders, and similar language model technology is not allowed. By submitting your solution, you legally declare to have solved the problems alone, without communicating with anybody, and not using language models.

You can use all functions provided in the included files, as well as functions that we implemented in the course. If the source is missing in this folder, you can include it in `Exam.scala`, so that things compile on our side. You can use the standard library functions. Staying closer to the course API is likely to yield better solutions.

There is a skeleton test file (`Exam.test.scala`) in the bundle, that you can use to test your solutions. It does not contain any useful tests. It is just there to get you started with testing faster. We will not grade or see the changes in this file. Do not hand it in.

We do not recommend writing and running tests if you are pressed for time. It is a good idea to run and test, if you have time.

Grading. We require that your hand-in compiles. The directory has a project setup, so compilation with `scala-cli` shall work out-of-the-box. If you cannot make a fragment compile, put your solution in a comment, next to the three question marks. We will grade the solutions in comments.

We will check whether the file compiles by running

```
scala-cli compile .
```

Hand-ins that do not compile will fail the exam, as will the hand-ins that are presented in imperative programming style.

The answers will be graded manually. We focus on the correctness of ideas, the use of concepts, clarity, and style. We will use undisclosed automatic tests during grading, but not to compute the final grade, but to help us debug your code.

Submission. Keep the solutions within 80 character columns to make grading easier. Do not modify `Exam.scala` in other ways than answering the questions or adding imports and source of needed functions. Do not reorder the answers, and do not remove question numbers or comments from the file.

Submit the completed `Exam.scala` file and only this file to LearnIT. Do not convert to any other format. Do not submit the entire zip archive. The only accepted file format is `.scala`

Hand-in: `Exam.scala`.

Good luck!

Root Mean Square Error

Question 1. Two streams of incoming real numbers are represented as two synchronized non-empty `LazyList[Double]`. The list `gs` contains ground truth values (g_i). The list `ps` contains the corresponding prediction values (p_i) from a machine learning model. We want to compute the *root mean square error* (RMSE) of predictions with respect to the ground truth. The definition of RMSE is:

$$\text{RMSE}(\texttt{gs}, \texttt{ps}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(g_i - p_i)^2}, \tag{1}$$

where n is the length of both lists. This means that a *root mean square error* is the root of a sum of square of differences between predictions and ground truths, divided by the length of the sequence.

Example:

$$\text{RMSE}(\texttt{LazyList(0.8, 2.2)}, \texttt{LazyList(1.0, 2.0)}) = \sqrt{1/2((0.8-1.0)^2 + (2.2-2.0)^2)} = \sqrt{1/2(0.04+0.04)} = \sqrt{0.04} = 0.2 \tag{2}$$

Implement RMSE assuming that both lazy lists are finite, same length, and non-empty. Do not use recursion.

Question 2. Implement a property-based test for Question 1. The property must check that the RMSE is always non-negative. As the test will be run on pairs of lazy lists, we need a given arbitrary instance that creates pairs of finite lists of equal lengths. Please include it in the solution. For full points, this instance should allow generating lists of arbitrary element type, not just `Double`.

Question 3. Now assume that both lists are infinite. Unfold a *running* RMSE, that is, a lazy list of RMSE values for each prefix of the input lists. Do not use direct recursion.

Example:

$$\text{runningRMSE}(\texttt{LazyList(0.8, 1.0, 2.2)}, \texttt{LazyList(1.0, 1.0, 2.0)}) \approx \texttt{LazyList(0.2, 0.14, 0.16)} \tag{3}$$

The first element is the RMSE of the first pair ((0.8) and (1.0)), the second element is the RMSE of the first two pairs. The nth element in the `LazyList` would be the RMSE of the first n pairs.

Question 4. Implement a property-based test for Question 3. The property should state that the running RMSE is a sequence of non-negative numbers. This should be checked for random pairs of finite lazy lists, but only for the prefix of 500 elements of the result.

You will need an arbitrary instance again to generate lists, but do not use the one from Question 2. Instead, `runningRMSEProperty` should obtain it from the caller, thanks to a context constraint.

Question 5. In order to run the test in Question 4, it would be useful to have a generator that creates random *infinite* lazy lists. Then we could test arbitrarily long prefixes. Implement such a generator of infinite lazy list as a function of type `Gen[LazyList[A]]`. Do not use it to test, just implement it.

Hint: A simple solution is obtained using `sequence` on `LazyList[Gen[A]]`. This version of `sequence` is non-trivial, but it is already provided with the exam. It behaves like `sequence` on strict lists, which we have seen in the course. Also, you will need to modify the signature of `gen5`, in order to handle any generate-able type `A`.

Lenses

Question 6. Provide a `Lens` between the `Gen` and `Arbitrary` types. Then respond to the question in the comment in English, on which lens laws your new lens satisfies. For each answer, provide a brief argument in English (1-2 lines).

Monads

Question 7. The `Exam.scala` file contains an implementation of binary trees, slightly different from the one that we created in the course. Using our old trees from the course is of no help in this exercise. Provide evidence that this new implementation is a functor, in the sense of `adpro.monad.Functor`.

Question 8. Implement a function `f8` that increments all values stored in a `Tree[Int]`. The function should *not* use your `Functor` instance. Instead, it should allow the caller to provide a `Functor` instance implicitly in context. (The caller could, for example, use your instance from Question 7, but perhaps they have another one they want to use).

Parsing

Question 9. Use our parser combinator library to implement a parser of trees labeled by integers. We use the type from the previous exercise as the abstract syntax. The concrete syntax is as follows:

- A leaf is represented by empty parentheses, i.e. `()`
- A branch is represented as parentheses containing three elements: an integer, a left sub-tree, and a right sub-tree (no separators). For instance, `Branch(42, Leaf, Leaf)` is written `(42()())`.
- All integer constants in the tree are unsigned. An integer constant is any non-empty sequence of decimal digits.
- No white space (and no new lines) appear in the input.

Given an input, the parser should produce a `Tree[Int]` value, or a failure.

Hint: Since the included parser combinators do not naturally backtrack, it is convenient to use the grammar in LL(1) form.

$$\begin{array}{lcl} \mathsf{T} & \rightarrow & \texttt{LPAR}\ (\mathsf{B} \mid \mathsf{L}) \\ \mathsf{B} & \rightarrow & \texttt{INT}\ \mathsf{T}\ \mathsf{T}\ \texttt{RPAR} \\ \mathsf{L} & \rightarrow & \texttt{RPAR} \end{array} \tag{4}$$